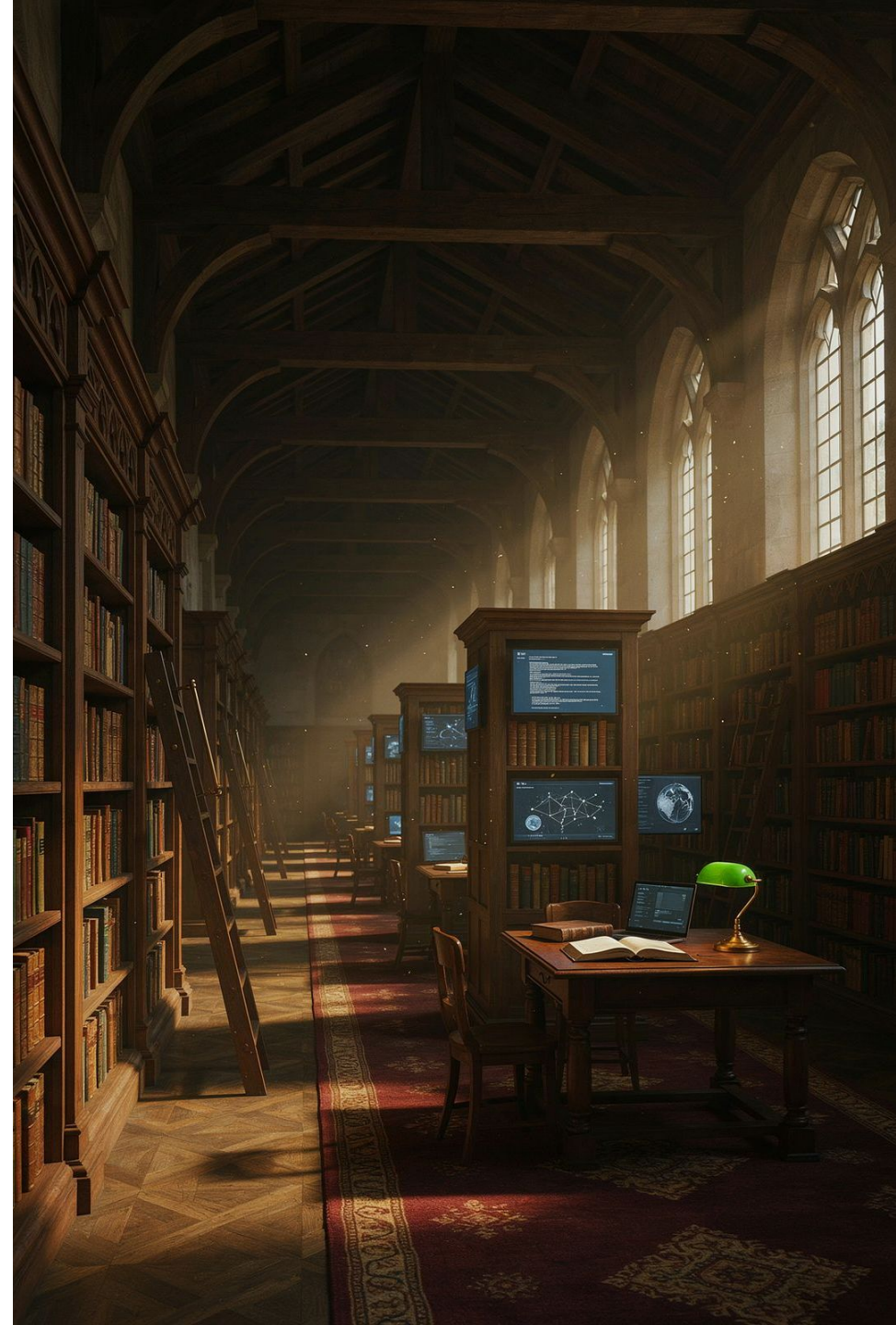


Lịch Sử Tiến Hóa Phương Pháp Phát Triển Phần Mềm

Từ Waterfall đến Agentic AI · và tại sao mọi thứ bạn biết về lập trình đang được viết lại

CHƯƠNG 1 · SDD BOOTCAMP · 2026



Tổng Quan Chương 1

Hành trình nửa thế kỷ tiến hóa — từ Waterfall đến kỷ nguyên AI Agents.



Waterfall (1970s)

Lên kế hoạch mọi thứ trước khi code



DevOps (2010s)

Tự động hóa mọi thứ có thể



Agile (2001)

Thay đổi là bình thường



AI Agents (2025–2026)

Ủy quyền code cho AI

Ba paradigm lớn được phân tích: **Vibe Coding** · **SDD** · **ADD** — mỗi paradigm gắn với ví dụ thực tế và trường hợp sử dụng cụ thể.

Điểm Trọng Tâm Của Chương

1 Vấn đề được giải quyết

Phương pháp này giải quyết vấn đề gì của thể hệ trước — và nó tạo ra vấn đề mới nào?

2 Lựa chọn đúng paradigm

Ai nên dùng paradigm nào — và trong bối cảnh nào? Không có công thức duy nhất cho mọi tình huống.

3 Tư duy kỹ sư hệ thống

Tiếp cận như kỹ sư có tư duy hệ thống, không học theo trào lưu. **Tools minh họa:** GitHub Copilot · ChatGPT · Claude · Cursor

Mục Lục Chương 1

01

1.1 · Từ Waterfall đến Agile

Nửa Thế Kỷ Cô Đặc — hành trình từ quy trình cứng nhắc đến văn hóa linh hoạt

03

1.3 · Vibe Coding & Nhu Cầu Cấu Trúc

Ba Paradigm Lớn — sức hút và nguy hiểm tiềm ẩn của Vibe Coding

02

1.2 · Sự Xuất Hiện Của AI

Trong Quy Trình Phát Triển — ba giai đoạn tiến hóa của AI coding tools

04

1.4 · Bức Tranh Toàn Cảnh 2026

SDD, ADD & Hybrid Approach — framework tối ưu cho nhóm và team nhỏ



⚡ Hands-on & Thực Tế: Ứng dụng thực tiễn cho đồ án và công việc xuyên suốt toàn chương

Từ Waterfall đến Agile



Vấn đề

Mỗi phương pháp ra đời để giải quyết vấn đề của phương pháp trước



Nửa thế kỷ

50+ năm tiến hóa liên tục từ 1970 đến nay



Tất yếu

Mỗi bước tiến là phản ứng với thực tế thay đổi của ngành phần mềm



1970 – 1990S

Waterfall: "Lên Kế Hoạch Mọi Thứ Trước Khi Code"

Năm 1970, **Winston Royce** xuất bản bài báo mô tả quy trình phát triển phần mềm tuần tự: Requirements → Design → Implementation → Verification → Maintenance. Mô hình Waterfall chỉ **chạy một chiều từ trên xuống**. Điều thú vị: chính Royce cũng cảnh báo mô hình này có nhiều rủi ro nghiêm trọng.

Waterfall — Khi Nào Hiệu Quả, Khi Nào Thất Bại

✓ Waterfall hoạt động tốt khi...

- Phần mềm giao trên đĩa mềm hoặc băng từ — thay đổi sau khi ship cực kỳ tốn kém
- Yêu cầu phần mềm tương đối ổn định (hệ thống quân sự, ngân hàng lõi)
- Cần quy trình rõ ràng, tài liệu hóa đầy đủ cho dự án quy mô lớn

× Vấn đề cốt lõi

- Khách hàng **không biết chính xác mình muốn gì** cho đến khi thấy sản phẩm chạy
- Khi phần mềm hoàn thành sau 2–3 năm, **yêu cầu đã thay đổi hoàn toàn**
- Sản phẩm trở nên lỗi thời ngay tại thời điểm bàn giao

Kế hoạch chi tiết là cần thiết, nhưng kế hoạch không thể thay đổi là nguy hiểm. Đây cũng là bài học sẽ quay lại khi chúng ta nói về Spec-Driven Development — viết spec tốt *không có nghĩa là "đông cứng" spec*.

2001 – 2015

Agile: "Thay Đổi Là Bình Thường"

Tháng 2 năm 2001, **17 nhà phát triển phần mềm** gặp nhau tại Utah và viết ra **Agile Manifesto** — một tài liệu chỉ vỏn vẹn 68 từ đã thay đổi toàn bộ ngành phần mềm. Thay vì lên kế hoạch mọi thứ từ đầu, Agile chấp nhận rằng yêu cầu sẽ thay đổi — và xây dựng quy trình *xung quanh* sự thay đổi đó.



4 Giá Trị Cốt Lõi Của Agile Manifesto

Con người & tương tác hơn là quy trình & công cụ	Phần mềm chạy được hơn là tài liệu đầy đủ
Hợp tác với khách hàng hơn là đàm phán hợp đồng	Phản hồi với thay đổi hơn là tuân theo kế hoạch

Các Framework Agile Phổ Biến



Scrum

Sprint 2 tuần, Daily standup, Sprint Review & Retrospective. Team tự tổ chức, tập trung giao sản phẩm cuối mỗi sprint.



Kanban

Công việc chảy liên tục qua các cột. Giới hạn WIP (Work In Progress) để tránh quá tải. Tối ưu throughput.



XP / Extreme Programming

Pair programming, Test-Driven Development (TDD), Continuous Integration. Tập trung cực đoan vào chất lượng kỹ thuật.



Agile giải quyết tốt "yêu cầu thay đổi" nhưng tạo ra thách thức mới: Làm thế nào để giao hàng liên tục mà không hy sinh chất lượng? → Câu trả lời dẫn đến **DevOps**.



2010S – 2022

DevOps & CI/CD: "Tự Động Hóa Mọi Thứ Có Thể"

DevOps là một **văn hóa**: xóa bỏ bức tường giữa Development và Operations. CI/CD trở thành tiêu chuẩn ngành — mỗi lần commit code, hệ thống tự động **build, test, và deploy**. Chu kỳ giao hàng từ hàng tháng rút xuống còn hàng giờ.

DevOps — Công Cụ & Đo Lường

Công cụ tiêu biểu

- Jenkins · GitLab CI · GitHub Actions — pipeline automation
- Docker · Kubernetes — container orchestration
- Terraform · Ansible — infrastructure as code

Google DORA Metrics

- **Deployment frequency** — tần suất deploy
- **Lead time** — thời gian từ code đến prod
- **Change failure rate** — tỷ lệ lỗi sau deploy
- **Recovery time** — thời gian phục hồi sự cố

📌 ⚡ **Bottleneck vẫn còn đó:** Dù CI/CD có nhanh đến mấy, nếu developer mất 3 ngày để viết một feature, pipeline vẫn phải chờ 3 ngày. Bottleneck lớn nhất vẫn là **con người viết code** — và đây là lúc AI bước vào cuộc chơi.

Timeline Tiến Hóa Phương Pháp Phát Triển Phần Mềm

Giai đoạn	Phương pháp	Giải quyết vấn đề	Tạo vấn đề mới
1970s	Waterfall	Quy trình rõ ràng cho dự án lớn	Không thích ứng được thay đổi
1990s	RUP / Iterative	Chia thành iterations nhỏ hơn	Quá nhiều documentation
2001	Agile / Scrum	Thích ứng nhanh, phản hồi sớm với thay đổi	Giao hàng liên tục = áp lực ops
2010s	DevOps / CI/CD	Tự động hóa deploy, giảm bottleneck ops	Bottleneck: tốc độ viết code
2022+	AI-Assisted	Tăng tốc viết code đột biến	Chất lượng, kiểm soát, trust?

Sự Xuất Hiện Của AI Trong Coding

20 năm

Agile mất bao lâu để mainstream

3 năm

AI coding tools đạt mainstream

84–92%

Developer toàn cầu dùng AI tools
(2026)

Tốc độ áp dụng chưa từng có tiền lệ trong lịch sử ngành phần mềm.

A close-up photograph of a person's hands typing on a black mechanical keyboard. Above the keys, there are translucent, glowing green suggestions of code words: 'fetchdata', 'fetch', 'fetch', 'fetch', and 'fetchnone'. In the background, a computer monitor displays lines of code in a dark-themed editor.

GIAI ĐOẠN 1 · 2021–2023

Autocomplete: "Gợi Ý Dòng Tiếp Theo"

GitHub Copilot ra mắt bản technical preview vào tháng 6/2021 và GA tháng 6/2022. Đây là công cụ đầu tiên mang AI vào IDE của hàng triệu developer dưới dạng một thứ rất quen thuộc: **autocomplete**. AI hoạt động ở mức dòng code và hàm, không hiểu dự án, không biết kiến trúc tổng thể, không đọc file khác ngoài file hiện tại.

Giống một người bạn ngồi cạnh biết rất nhiều pattern code, nhưng không biết gì về dự án bạn đang làm.



GIAI ĐOẠN 2 · 2023–2024

Chat-Based: "Hỏi AI, Nhận Code"

ChatGPT ra mắt tháng 11/2022 và thay đổi mọi thứ. Lần đầu tiên, developer có thể mô tả vấn đề bằng **ngôn ngữ tự nhiên** và nhận lại một khối code hoàn chỉnh. Tiếp theo: Claude (Anthropic), Gemini (Google), Cursor (2023) — mỗi công cụ mở rộng thêm khả năng hiểu context.

Chat-Based AI — Đặc Điểm & Giới Hạn

Workflow điển hình

1. Hỏi bằng ngôn ngữ tự nhiên
2. Nhận code từ AI
3. Copy về IDE → Test
4. Không chạy → Hỏi lại
5. Vòng lặp copy-paste tiếp tục

⚠ Giới hạn cần vượt qua

- AI hoạt động ở mức **khối code / file**, không phải toàn dự án
- Context = những gì bạn paste vào — rất hạn chế
- Không tự chạy code, không kiểm tra kết quả
- Không đọc toàn bộ codebase để hiểu kiến trúc

Vòng lặp hỏi-đáp thủ công tạo ra ma sát đáng kể trong quá trình phát triển.



GIAI ĐOẠN 3 · 2025-NAY

Agentic: "AI Tự Làm, Bạn Duyệt"

Bước nhảy lớn nhất xảy ra từ đầu 2025. **Claude Code** (Anthropic, 02/2025), GitHub Copilot Agent Mode, Cursor Agent Mode, OpenAI Codex CLI — tất cả chuyển từ mô hình "hỏi-đáp" sang mô hình "**ủy quyền**". AI không còn chỉ gợi ý — nó tự lên kế hoạch, tự thực thi, và tự kiểm tra kết quả.

Đặc Điểm Của Agentic AI Coding

Phạm vi toàn dự án

Tự đọc cả codebase, hiểu kiến trúc, biết file nào liên quan đến file nào — không chỉ nhìn vào file đang mở.

Thực thi độc lập

Có thể chạy terminal commands, tự kiểm tra kết quả, sửa lỗi compile-time và runtime trong một phiên làm việc duy nhất.

Workflow mới: Mô tả → Duyệt

Bạn mô tả mục tiêu → Agent tự lên kế hoạch → Agent tự edit nhiều file, chạy test, sửa lỗi. Human review là bước cuối cùng.

TOOLS TIÊU BIỂU 2025

Claude Code · Cursor Agent Mode · GitHub Copilot Agent · OpenAI Codex CLI · Windsurf

10x

Engineer truyền thống

100x

Engineer điều phối AI agents

Thực tế đã chuyển từ *bổ trợ code* sang *ủy quyền code*. 10x engineer có thể trở thành 100x engineer — không bằng cách viết nhiều code hơn, mà bằng cách điều phối nhiều agent hơn.

— Dev Community · "The AI Revolution in 2026"

Ba Giai Đoạn Tiến Hóa Của AI Coding

Tiêu chí	Autocomplete	Chat-Based	Agentic	Xu hướng 2026+
Phạm vi	Dòng / hàm	Khối code / file	Toàn dự án	Multi-dự án
Workflow	Gõ → Tab	Hỏi → Copy → Paste	Mô tả → Duyệt	Spec → Auto
Context	File hiện tại	Những gì bạn paste	Cả codebase	Codebase + MCP
Kiểm soát	Từng dòng	Từng khối	Từng task	Từng spec
Rủi ro chính	Thấp	Copy sai context	Agent "tự ý"	Over-reliance

Mức Độ Áp Dụng AI Trong Lập Trình

84%

Đang dùng hoặc có kế hoạch dùng AI tools

Stack Overflow Developer Survey 2025

90%

Chuyên gia dùng AI trung bình 2 giờ/ngày

Google DORA Report 2025

92%

US developers dùng AI coding tools hàng ngày

Industry Report 2026

25%

Startups YC W2025 có 95% codebase do AI tạo

Y Combinator Winter 2025



⚠️ **Cảnh báo cho sinh viên:** Nếu làm đồ án SE mà không dùng AI tools, bạn đang ở vị trí bất lợi so với 84–92% developer toàn cầu. Nhưng **dùng AI không đúng cách còn nguy hiểm hơn không dùng.**

🏆 ICPC World Finals 2025

Hệ thống AI của OpenAI đạt **12/12 điểm tối đa**, xếp thứ nhất nếu là đội người — cột mốc lịch sử trong lập trình thi đấu.

⚡ Tốc độ áp dụng kỷ lục

GitHub Copilot đạt **1 triệu người dùng** nhanh hơn bất kỳ developer tool nào trong lịch sử. Không có tiền lệ để so sánh.

Vibe Coding & Nhu Cầu Cấu Trúc

Vibe Coding

Nhanh nhưng nguy hiểm — thiếu cấu trúc, tích lũy technical debt

Spec-Driven (SDD)

Đặc tả trước, code sau — chất lượng có kiểm soát

Agent-Driven (ADD)

AI tự thực thi — con người định hướng và kiểm soát

Vibe Coding — Định Nghĩa & Sức Hút

Người định nghĩa

Andrej Karpathy — đồng sáng lập OpenAI, cựu giám đốc AI tại Tesla

Thời điểm

Tháng 2/2025 — thuật ngữ lan rộng toàn ngành chỉ trong vài tuần

"Hoàn toàn buông theo cảm xúc, đón nhận sự tăng trưởng theo cấp số nhân, và quên rằng code thậm chí đang tồn tại."

— Andrej Karpathy, tháng 2/2025



Tại Sao Vibe Coding Hấp Dẫn Đến Vậy?



Cảm giác "siêu năng lực"

Tạo app hoàn chỉnh trong vài giờ. Prototype nhanh chưa bao giờ dễ đến vậy — không cần setup phức tạp, không cần đọc docs.



Hackathon sôi động hơn

Viết prompt → Nhận code từ AI → Chạy thử → Thấy chạy được → Tiếp tục. Chu kỳ feedback cực ngắn, cực kỳ thỏa mãn.



Phù hợp deadline ngắn

Đồ án ngắn hạn với áp lực deadline thực sự hưởng lợi từ tốc độ này — trong *ngắn hạn*. Vấn đề chỉ xuất hiện về sau.

Vấn Đề 1: Code Complexity Bùng Nổ

+41%

Tăng độ phức tạp code

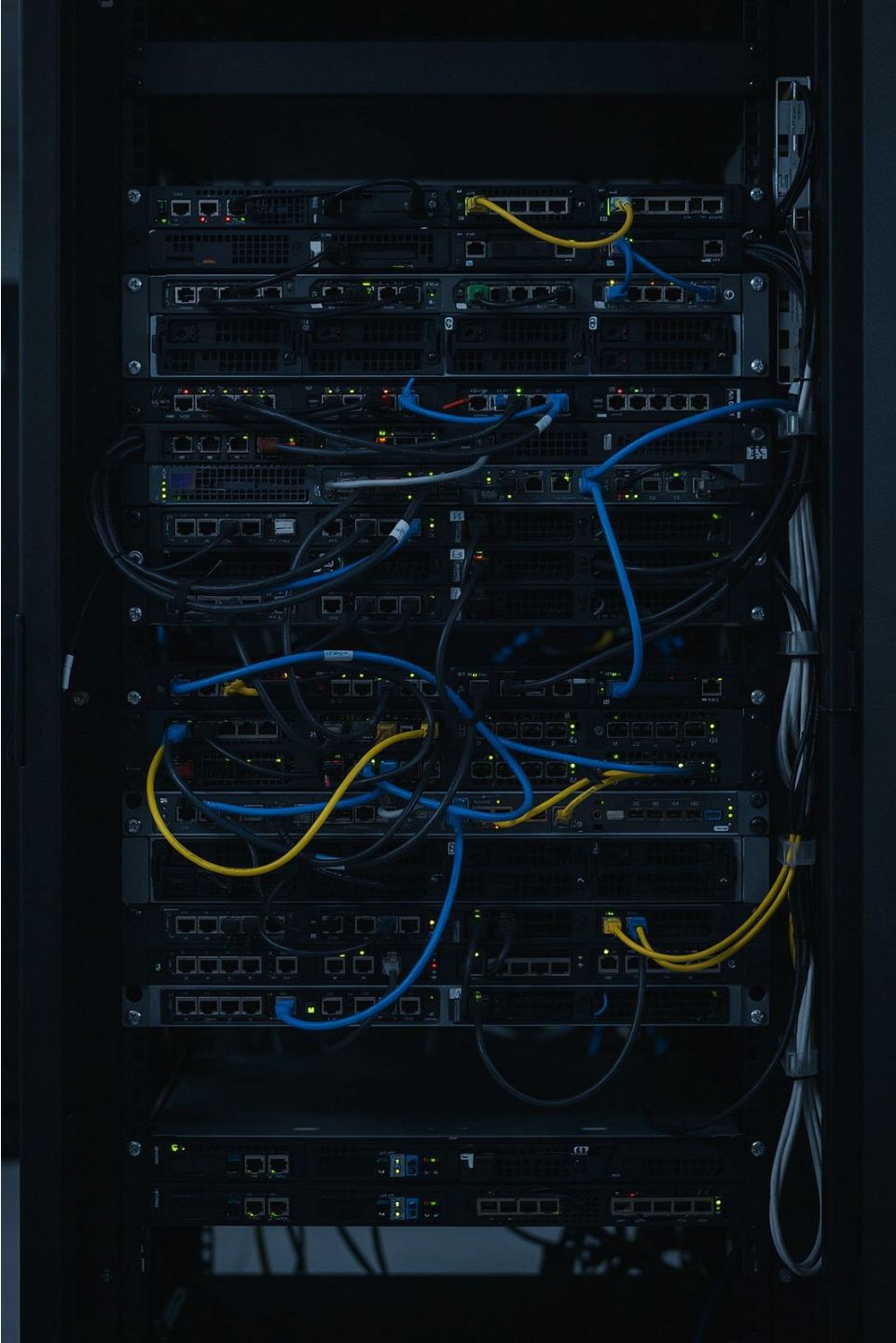
AI-assisted coding làm tăng code complexity đo được qua static analysis

+30%

Cảnh báo static analysis

Số cảnh báo từ code analyzer tăng đáng kể so với code viết tay

AI tạo code dài hơn, nhiều abstraction hơn, dùng nhiều dependencies hơn cần thiết. Sau vài tuần phát triển với Vibe Coding, **không ai trong nhóm hiểu toàn bộ codebase** — ngay cả người tạo ra nó.



Vấn Đề 2: Bảo Mật Bị Bỏ Qua

42%

Tỷ lệ lỗi bảo mật cao nhất

LLM tạo code có lỗi bảo mật từ 9.8% đến 42.1% tùy benchmark và loại task

93%

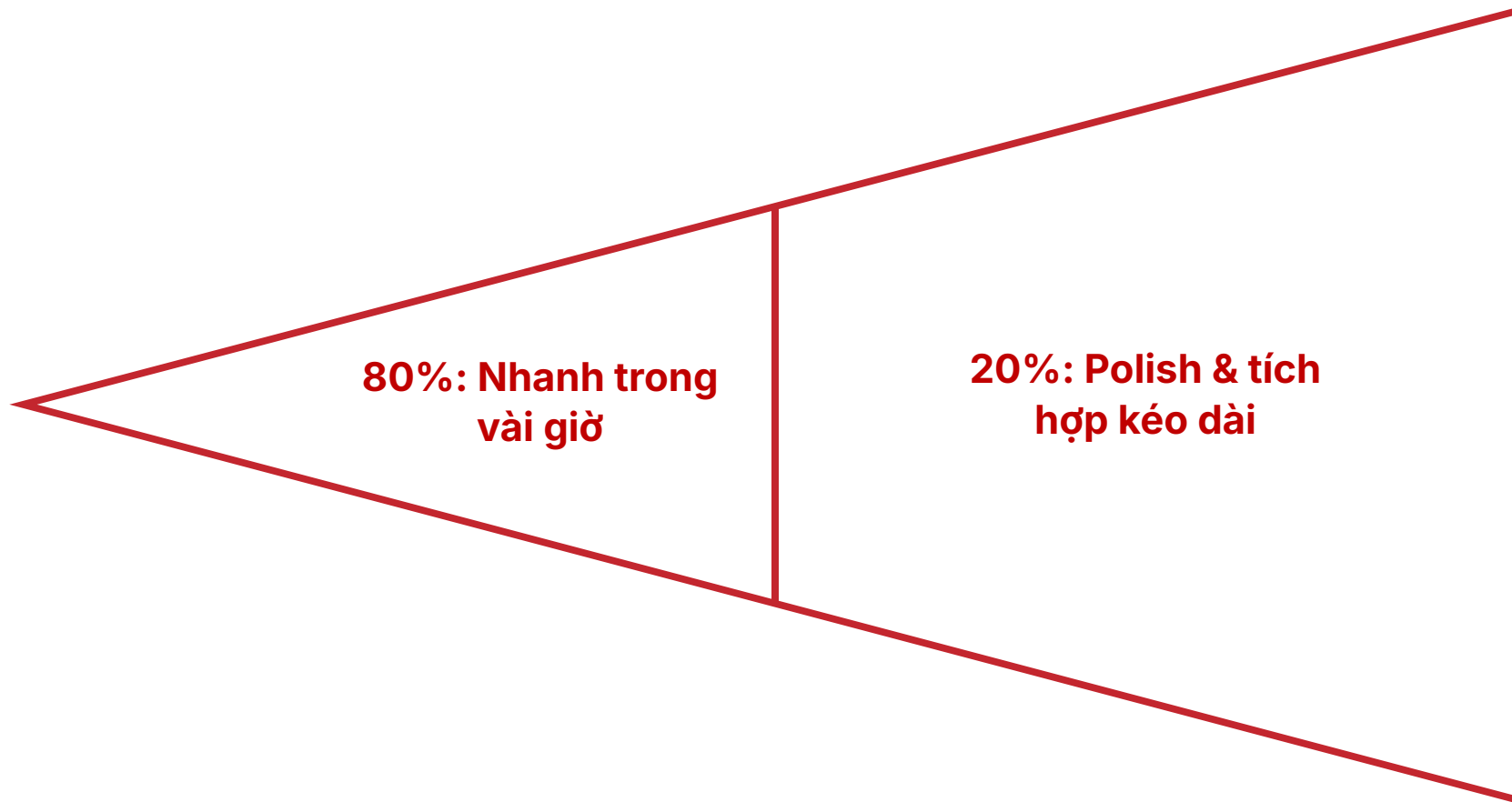
Lỗi mức Blocker/Critical

56–93% các lỗi bảo mật được phân loại mức Blocker hoặc Critical — không thể bỏ qua

⊗ AI **không chủ động nghĩ về bảo mật** trừ khi được yêu cầu rõ ràng trong prompt. Khoảng 2% tổng số issues trong code AI-generated là lỗi bảo mật — tỷ lệ cao một cách đáng báo động.



Vấn Đề 3: "Final 20% Problem"



AI có thể build 80% ứng dụng trong vài giờ — nhưng **20% còn lại** (edge cases, polish, integration, performance optimization) lại mất thời gian bằng cả dự án trước đây. 15.000 dòng code mà không ai hiểu = cơn ác mộng khi cần sửa.

Vấn Đề 4: Technical Debt Tích Lũy

Triệu chứng điển hình

- Naming không nhất quán giữa các file
- Cùng một vấn đề giải quyết theo 3 cách khác nhau trong 3 file
- Patterns lẫn lộn không có nguyên tắc
- Code tối ưu cho "chạy được ngay", không phải "dễ bảo trì"

Khi nào nổ ra?

Tuần **15–20** của đồ án 20 tuần là thời điểm khủng hoảng điển hình — khi cần tích hợp các module, viết test cuối, và demo cho giáo viên. Đây là lúc technical debt tích lũy trở thành thảm họa thực sự.

"Tốc độ là thật... nhưng founder nào chưa từng học cách quản lý context sẽ gặp rắc rối lớn khi AI tạo ra 15.000 dòng code mà họ không hiểu."

— *Reddit · r/programming, 2025*

Trích dẫn quan trọng

Tháng 9/2025, Fast Company xuất bản bài "**The Vibe Coding Hangover Has Arrived**" — các senior software engineer mô tả trải nghiệm làm việc với code do AI tạo mà không ai hiểu được. Đây là lời cảnh tỉnh cho toàn ngành.

+41% Code Complexity

Độ phức tạp tăng đột biến, khó maintain

42% Security Vulnerabilities

Tỷ lệ lỗi bảo mật đáng báo động

Final 20% Trap

80% nhanh, 20% cuối mất nhiều thời gian nhất

Vibe Coding vs. Structured AI Development

Tiêu chí	Vibe Coding	Structured (SDD + ADD)
Triết lý	Prompt → Code → Ship	Spec → Plan → Code → Validate
Tốc độ ban đầu	Rất nhanh (giờ)	Chậm hơn (ngày)
Tốc độ lâu dài	Chậm dần (nợ kỹ thuật)	Duy trì ổn định
Chất lượng code	Không đảm bảo	Có validation gates
Bảo mật	Phụ thuộc may rủi	Spec định nghĩa constraints
Team collaboration	Khó (ai hiểu code?)	Dễ (spec là ngôn ngữ chung)
Phù hợp cho	Prototype, hackathon, 1 người	Đồ án thực, team, sản phẩm



Bẫy Vibe Coding Cho Nhóm Đồ Án

⚠️ ⚠️ **Cảnh báo cho nhóm đồ án:** Nhiều nhóm sinh viên bắt đầu bằng Vibe Coding vì nó nhanh, rồi gặp khủng hoảng ở giữa kỳ khi cần tích hợp, test, và demo cho giáo viên.

Playbook này giúp bạn tránh bẫy đó bằng cách **dành 2–3 tuần đầu để setup đúng**, và thu hoạch tốc độ suốt 17 tuần còn lại. Đầu tư upfront vào spec và kiến trúc là khoản đầu tư có ROI cao nhất trong toàn bộ dự án.

Bức Tranh Toàn Cảnh 2026

Spec-Driven Development (SDD)

Triết lý: Đặc tả rõ ràng trước khi code. Phù hợp cho kiến trúc, API, database schema.

Agent-Driven Development (ADD)

Triết lý: AI là collaborator chủ động. Phù hợp cho implementation, testing, deployment.

📌 Hai approach bổ sung cho nhau hoàn hảo → **Hybrid Approach** là lựa chọn tối ưu.



APPROACH 1

Spec-Driven Development (SDD)

"Vấn đề không phải AI kém, mà là chúng ta đưa cho AI input kém." Nếu bạn viết một specification rõ ràng, chi tiết, và có thể kiểm chứng được, thì AI sẽ tạo ra code chất lượng cao hơn nhiều.

SDD — Nguyên Tắc & Đánh Giá

Nguyên tắc core

- **Spec là artifact chính**, code là sản phẩm dẫn xuất
- Quy trình: Specify → Plan → Tasks → Implement → Validate
- Tools tiêu biểu: **GitHub Spec Kit**, **Amazon Kiro**

Đánh giá khách quan

- ✓ Reproducible, quality gates rõ ràng
- ✓ Team alignment — spec là ngôn ngữ chung
- ✓ Dễ bàn giao, dễ kiểm tra với giáo viên

△ Overhead viết spec ban đầu

△ Nguy cơ over-specification

△ Chậm hơn cho task nhỏ và đơn giản

APPROACH 2

Agent-Driven Development (ADD)

"AI đã đủ thông minh để tự làm, chỉ cần chỉ đường đúng." Thay vì viết spec chi tiết, hãy cung cấp context đầy đủ rồi để agent tự lên kế hoạch và thực thi. Human là nhạc trưởng, AI là dàn nhạc.



ADD — Nguyên Tắc & Đánh Giá

Nguyên tắc core

- **Agent là collaborator**, không phải tool bị động
- Context setup gồm **AGENTS.md**, constraints, domain knowledge
- Quy trình: Context Setup → Intent → Agentic Execution → Human Review
- Tools: **Claude Code, Cursor Agent Mode, Codex CLI**

Đánh giá khách quan

- ✓ Nhanh, flexible, linh hoạt với thay đổi
- ✓ Explore codebase hiệu quả
- ✓ Xử lý tốt boilerplate và repetitive tasks
- △ Context drift khi task dài
- △ Hallucination, tự ý thay đổi không mong muốn
- △ Khó kiểm soát với team lớn

★ RECOMMENDED

Hybrid Approach: Kết Hợp Tốt Nhất Của Cả Hai

Nguyên tắc cốt lõi: Dùng **SDD** cho những gì cần **ĐÚNG TỪ ĐẦU** (kiến trúc, API contracts, data models, security). Dùng **ADD** cho những gì cần **NHANH VÀ ITERATIVE** (UI components, business logic, tests, boilerplate).



Hai Tầng Của Hybrid Approach

SDD Layer — Architecture

- Kiến trúc hệ thống → **Full spec**
- API contracts → **Full spec**
- Database schema → **Full spec**
- Security constraints → **Full spec**

Những quyết định này tốn kém nếu sai — phải đúng từ đầu.

ADD Layer — Implementation

- UI components → **Agent tự do**
- Unit tests → **Agent tự do**
- Bug fixing → **Agent tự do**
- Boilerplate / scaffold → **Multi-agent**

Những task này lặp đi lặp lại và dễ kiểm tra — để agent tự do tăng tốc.

📌 **Spec ở tầng kiến trúc, Agent ở tầng implementation.** Đây là nguyên tắc phân chia trách nhiệm rõ ràng nhất trong Hybrid Approach.

Khi Nào Dùng SDD, Khi Nào Dùng ADD?

Loại task	Spec Depth	Agent Autonomy	Approach
Kiến trúc hệ thống	Full spec	Guided	SDD
API contracts	Full spec	Guided	SDD
Database schema	Full spec	Agentic	Hybrid
Feature logic (core)	Standard spec	Agentic	Hybrid
UI components	Light spec	Agentic	ADD
Unit tests	None	Agentic	ADD
Bug fixing	None	Agentic	ADD
Refactoring	Light spec	Agentic	Hybrid

⚠️ Hybrid không phải là "dùng cả hai lung tung". Đây là framework có nguyên tắc rõ ràng — khi nào cần spec chặt, khi nào để agent tự do.

Hybrid Phù Hợp Cho Nhóm Sinh Viên: 5 Lý Do

Với nhóm 5 người làm đề án 20 tuần, Hybrid mang lại những lợi ích cụ thể và đo lường được:



1 Spec làm ngôn ngữ chung của nhóm

Khi 5 người cùng code, spec rõ ràng giúp tránh hiểu lầm và code mâu thuẫn giữa các module.

2 Agent tăng tốc viết code

Mỗi thành viên hoàn thành nhiều hơn trong cùng thời gian, đặc biệt khi vừa học vừa làm.

3 Validation gates bảo vệ chất lượng

Spec-based review phát hiện lỗi sớm, trước khi lan sang module khác và trở thành technical debt.

4 Dễ phân công và theo dõi

Mỗi người assign spec + tasks rõ ràng — giáo viên đánh giá đóng góp cá nhân dễ dàng hơn.

5 Chuẩn bị cho career thực tế

SDD và ADD đều là kỹ năng đang được tìm kiếm trên thị trường lao động 2026.

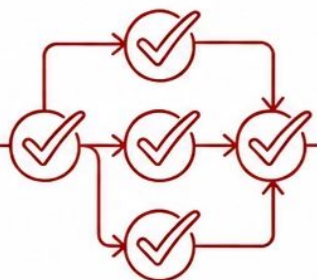
Infographic: Ba Paradigm Và Lựa Chọn Của Bạn

LẬP TRÌNH THEO CẢM HỨNG (VIBE CODING)



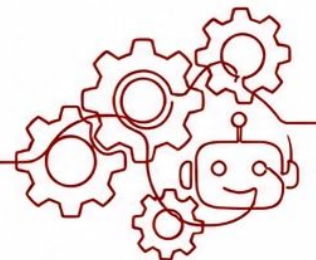
- Khởi đầu nhanh chóng
- Rủi ro cao
- Dành cho hackathon, nguyên mẫu

LẬP TRÌNH DỰA TRÊN ĐẶC TẢ



- Các công chất lượng có cấu trúc
- Đặc tả rõ ràng
- Phù hợp cho các nhóm, sản phẩm

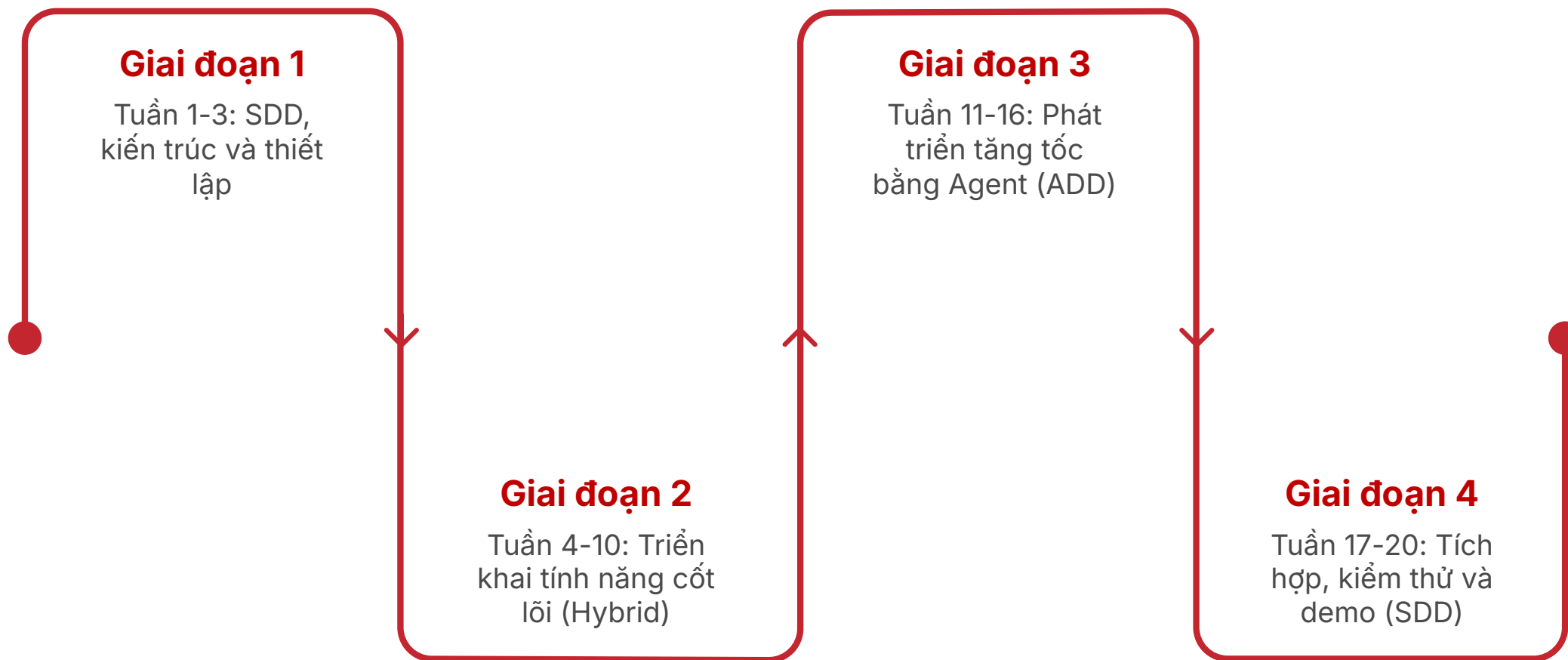
LẬP TRÌNH DỰA TRÊN AGENT



- Thực thi tự động
- Linh hoạt
- Dành cho các tác vụ triển khai

Mỗi paradigm phù hợp với một bối cảnh cụ thể. Kỹ sư giỏi không chọn một paradigm rồi dùng mãi — họ chọn đúng paradigm cho đúng tình huống.

Lộ Trình Hybrid: Phân Bỏ 20 Tuần Đồ Án



Đầu tư nặng vào SDD ở giai đoạn đầu tạo nền tảng vững chắc để ADD phát huy tối đa tốc độ ở giai đoạn giữa. Giai đoạn cuối quay lại SDD để đảm bảo chất lượng trước khi demo.

Key Takeaways



Tiến hóa liên tục

Waterfall → Agile → DevOps → AI Agents — mỗi bước giải quyết vấn đề của bước trước



Vibe Coding có giới hạn

Nhanh nhưng tạo technical debt, bảo mật yếu, và "Final 20% Problem"



Hybrid là tối ưu

SDD cho kiến trúc + ADD cho implementation = chất lượng và tốc độ

5 Bài Học Cốt Lõi Của Chương 1

→ Vòng lặp tiến hóa không có điểm dừng

Phương pháp phát triển phần mềm tiến hóa liên tục — mỗi bước mới giải quyết vấn đề của bước trước và tạo ra thách thức mới.

→ AI coding: 3 giai đoạn, chúng ta đang ở đầu Agentic

Autocomplete → Chat → Agentic. Giai đoạn Agentic mới bắt đầu — những ai hiểu cách điều phối agent sẽ có lợi thế lớn.

→ Vibe Coding: nhanh ngắn hạn, đắt dài hạn

Code complexity +41%, security risks cao, "Final 20% Problem" — những cái bẫy thực tế có thể phá hỏng đồ án ở giai đoạn cuối.

→ SDD + ADD: hai mặt của một đồng tiền

SDD giải quyết chất lượng bằng specification · ADD giải quyết tốc độ bằng agent autonomy. Cần cả hai.

→ Hybrid là approach tối ưu cho nhóm nhỏ

SDD ở tầng kiến trúc + ADD ở tầng implementation — không dùng lung tung mà có nguyên tắc rõ ràng.

Kiểm Tra Hiểu Biết — Câu Hỏi Tự Học

1

Tại sao Waterfall thất bại?

Điều gì đã thay đổi trong môi trường phần mềm từ 1970 đến 2001 khiến Waterfall không còn phù hợp?

2

Bottleneck trong DevOps là gì?

Sau khi CI/CD tự động hóa deployment, bottleneck lớn nhất còn lại ở đâu — và AI giải quyết nó thế nào?

3

Khi nào Vibe Coding hợp lý?

Có tình huống nào mà Vibe Coding là lựa chọn tốt nhất không?
Và điều kiện để dùng an toàn là gì?

4

Điểm mạnh của Hybrid Approach?

Tại sao việc kết hợp SDD và ADD lại tốt hơn việc chỉ dùng một trong hai cho toàn bộ dự án?

Tiếp Theo: Chương 2

CHƯƠNG 2

Từ "Người Viết Code" Sang "Kiến Trúc Sư + Nhạc Trưởng"

Mô hình tư duy mới của developer trong kỷ nguyên AI — khi vai trò con người dịch chuyển từ thực thi sang định hướng và kiểm soát chất lượng.